

AI Code Review & Security Analysis Report

Professional Code Quality & Security Assessment

Generated: 2026-08-14 09:54 UTC | Language: python | Model: llama-3.3-70b-versatile

Executive Summary

Code Health Score	Risk Score	Total Findings
0/100	42	12

Merge Readiness: unknown

PR summary generation was skipped because Groq rate limited the request.

Severity Breakdown

Critical	High	Medium	Low
8	3	0	1

Detailed Findings

Type	Severity	Line	Message	Recommendation
1. Hardcoded Password	● High	1	password = "admin123"	Store passwords in environment variables.
2. Hardcoded API Key	● High	3	api_key = "123456789"	Store secrets securely.
3. SQL Injection	● Critical	7	query = "SELECT * FROM users WHERE name='" + name + "'"	Use parameterized queries.
4. Dangerous Dynamic Execution	● Critical	9	eval(input())	Avoid dynamic code execution.
5. Dangerous Dynamic Execution	● Critical	11	exec("print('Hello')")	Avoid dynamic code execution.
6. Command Injection	● Critical	15	subprocess.run("dir", shell=True)	Do not use shell=True with user input.

Type	Severity	Line	Message	Recommendation
7. Security	● Critical	1	Hardcoded sensitive information	Store sensitive information securely using environment variables or a secrets manager
8. Security	● Critical	5	SQL injection vulnerability	Use parameterized queries or prepared statements to prevent SQL injection
9. Security	● Critical	7	Arbitrary code execution via eval	Avoid using eval and instead use safer alternatives such as ast.literal_eval or json.loads
10. Security	● Critical	9	Arbitrary code execution via exec	Avoid using exec and instead use safer alternatives such as subprocess.run with a list of arguments
11. Security	● High	11	Command injection vulnerability via subprocess	Use subprocess.run with a list of arguments instead of shell=True to prevent command injection
12. Best Practice	● Low	3	Input validation is missing	Validate user input to prevent unexpected behavior

■ Key Recommendations

1. Address all Critical and High severity findings before merging.
2. Run additional security scanning tools to complement this analysis.
3. Consider implementing automated testing to catch similar issues in the future.

■ Code Quality Metrics

Overall Health Score: 0/100

Risk Level: Unknown

Maintainability: N/A

Security Score: N/A

■ Source Code Snapshot

```
password = "admin123"
```

```
api_key = "123456789"
```

```
name = input()
```

```
query = "SELECT * FROM users WHERE name='" + name + "'"
```

```
eval(input())
```

```
exec("print('Hello')")
```

```
import subprocess
```

```
subprocess.run("dir", shell=True)
```



This report was generated by the AI Code Review & Security Analysis Agent. Review all findings carefully before implementing changes.

Report generated on 2026-08-14 09:54 UTC